Modifying a Region of an Orphan Mesh and Re-Running the Analysis (Abaqus/CAE 2020+)

Goal: Replace a local region of an existing orphan-mesh model with a newly meshed part, tie the new region back to the original mesh, and re-run the analysis — while keeping the new region's node/element labels starting at **20001+** (offset of 20,000) so they never collide with the original mesh.

Your setup (as confirmed):

- New region mesh is **slightly denser** than the original orphan mesh → the new region will be the **secondary (slave)** surface in the tie.
- New region uses the **same element type and order** as the surrounding mesh.
- Abaqus **2020 or newer** → terminology is **main** / **secondary** (older versions say master / slave).

> **Back up first.** Copy your original `.inp` and save the `.cae` under a new name before you start deleting elements. Element deletion in an orphan mesh is destructive.

Overview of the flow

1. Import the original orphan-mesh model (`.inp`).
2. Import / mesh the new part with the modified region (already done).
3. Trim the new part down to only the modified region (already done

— partitioned + extra cells deleted).
4. **Renumber the new region's mesh to start at 20001+** (two options below).
5. In Assembly: delete the elements of the original orphan mesh in the region being replaced.
6. Instance the new region and position it into the gap.
7. Create surfaces on both sides and **tie** them (new = secondary).
8. Regenerate sets/loads/BCs on the exposed original faces if needed.
9. Write the input, sanity-check it, and submit.

Step 1 — Import the original orphan-mesh model

`File > Import > Model...` and select the original `.inp`. Abaqus creates a model containing the orphan mesh as a **part** (or as an assembly of orphan-mesh instances), together with the sets, surfaces, sections, interactions, step, loads, and BCs defined in the deck.

Note which part/instance holds the mesh region you intend to modify, and confirm the **highest node and element label** currently in use — this justifies the 20,000 offset. Quick way: open the `.inp` in a text editor and look at the top of the `*Node` and `*Element` blocks (labels are usually monotonically increasing), or in CAE use `Tools > Query > ...` / the mesh statistics. As long as the original max label is below 20000, the offset guarantees no collision.

Step 2 — Import and mesh the new part *(done)*

You imported the new geometry, meshed it, partitioned the modified region, and deleted the cells that were identical to the original. Confirm:

- The element type/order matches the original region (Mesh module

→ `Mesh > Element Type...`), since a matching interface behaves best across a tie.
- Mesh quality is acceptable (`Mesh > Verify`).

Step 3 — Renumber the new region to start at 20001+

Pick **one** of the two methods. Do this **before** you tie things together, so every set/surface you create on the new region already carries the offset labels.

Option A — Inside Abaqus/CAE (kernel command, no external file)

The Mesh module has no "starting label" field in the GUI, but the scripting kernel has a direct renumber call. This works on an **orphan mesh part**, so first convert the new meshed part to one if it isn't already:

1. Mesh module, with the new part active: `Mesh > Create Mesh Part`. This produces an orphan mesh part (e.g. `NewPart-mesh`).
2. Open the kernel command line interface at the bottom of CAE (the `>>>` prompt) and run:

   ```
   p = mdb.models['Model-1'].parts['NewPart-mesh']
   p.renumberNode(startLabel=20001, increment=1)
   p.renumberElement(startLabel=20001, increment=1)
   ```

   Replace `'Model-1'` and `'NewPart-mesh'` with your actual model/part names.

> **Verify the call signature in your version.** In most 2020+ releases the arguments are `startLabel` and `increment`; some releases also expose an `offset`-style relabel. If the kernel rejects the keywords, check `Help > ...` for `Part` object methods `renumberNode` / `renumberElement`, or fall back to Option B. There

> is also a GUI equivalent for orphan-mesh parts under the Mesh menu (renumber node/element labels) whose exact wording varies by release — the kernel command above is the version-robust route.

After renumbering, `Tools > Query > Node/Element` on the new part should report labels ≥ 20001.

Option B — MATLAB script on an exported `.inp` (deterministic)

This is the most controllable route and matches a text-based workflow.

1. Export **only the new region's mesh** to its own input file. Simplest way: with the orphan mesh part (from step A.1) instanced by itself in a scratch model, `Job > Write Input`, or export the part's mesh. The key requirement is that the file contains **only** the new part — no original orphan mesh — because the script shifts *everything* by the offset uniformly.
2. Run the provided script `renumber_inp_offset.m`:

   ```
   renumber_inp_offset('NewPart.inp', 'NewPart_20000.inp
   ```

3. Import the renumbered file back: `File > Import > Model...` (or import as a part / merge into the model), giving you the new region with labels starting at 20001.

What the script shifts (uniformly, +20000): every `*Node` label; every `*Element` id *and* its connectivity node ids; every id listed under `*Nset` / `*Elset`; and the `start,end` fields of `... , generate` sets (the increment is kept). **What it leaves alone:** set/surface *names*, node coordinates, sections, materials, and all other keywords. Because the whole file shifts together, connectivity and set references stay internally consistent.

> Do **not** run the script on the fully-assembled model deck — that would also shift the original orphan mesh. Run it only on the standalone new-part file.

Step 4 — Delete the original elements in the region being replaced

In the Mesh module (or via the Edit Mesh toolset on the orphan mesh part):

1. **Before deleting, freeze the region as named sets.** Select the region's elements and save them as an element set (e.g. `DEL_REGION_ELEMS`); select its nodes and save them as a node set (e.g. `DEL_REGION_NODES`). Abaqus writes these into the `.inp`, giving you an exact, permanent record of every label you are about to remove. This is essential for thermal/convection runs — see **Step 11 — Tracking deleted & added elements and nodes** below.
2. `Mesh > Edit...` → category **Element**, method **Delete**.
3. Select the elements of the original orphan mesh that occupy the region you're replacing (reuse `DEL_REGION_ELEMS` so the selection is repeatable and documented).
4. Delete them. This exposes a set of free faces on the remaining original mesh — these become the **main (master)** tie surface.

> Deleting elements can orphan nodes and can invalidate any set/surface/section that referenced the removed elements. Note which sets touched that region; you'll re-check them in Step 8.

Step 5 — Instance the new region and position it

Assembly module:

1. `Instance > Create` → select the renumbered new region part. Use a **dependent** instance (mesh comes from the part) unless you have a reason to mesh independently.
2. Position it into the gap: `Instance > Translate` / `Instance > Rotate`, or the constraint tools (`Constraint > Coincident`

`Point`, `Parallel Face`, etc.), or `Instance > Translate To` using matching vertices. The new region should sit in the void with its faces close to (ideally coincident with) the exposed original faces.
3. Confirm there's no interpenetration and the gap is small — a tie tolerates a small gap but large gaps distort load transfer.

Step 6 — Create the tie surfaces

You need two surfaces: one on the exposed original mesh, one on the new region.

1. `Tools > Surface > Create` → **main** surface = the exposed free faces of the original orphan mesh (the faces revealed by Step 4).
2. `Tools > Surface > Create` → **secondary** surface = the mating faces of the new (denser) region.

For orphan/element meshes, pick faces by element face selection (or reuse an element set + free-face definition). Name them clearly, e.g. `ORIG_TIE_MAIN` and `NEW_TIE_SEC`.

Step 7 — Create the tie constraint

Interaction module → `Constraint > Create > Tie`.

- **Main (master):** the original orphan-mesh surface (`ORIG_TIE_MAIN`).
- **Secondary (slave):** the new, denser region surface (`NEW_TIE_SEC`).

Why this assignment: the tie forces the secondary surface's nodes to follow the main surface. The **finer/denser mesh should be the secondary** so its many nodes are each constrained to the coarser main surface — this is the standard recommendation and gives the smoothest load transfer. Since your new region is the denser side, it is the

secondary. (If it were the other way around, coarser-as-secondary can miss detail and produce a lumpy interface.)

Recommended tie options:

- **Discretization:** *Surface to surface* (smoother stress transfer than node-to-surface across a solid interface).
- **Position tolerance:** start with the default; if some secondary nodes are excluded, increase it just enough to capture the intended faces (and no more, so you don't grab unintended nodes).
- **Adjust secondary surface initial position:** *On* — snaps secondary nodes onto the main surface to close any small initial gap, avoiding spurious initial strain.
- **Tie rotational DOFs:** not applicable for solid (C3D) elements — solids have no rotational DOF. (Only relevant if either side is a shell/beam.)

Because both sides are the **same element type and order**, no special compatibility handling is needed. The interface won't be node-coincident (densities differ), which is exactly why a tie — not a node merge — is the right tool.

Step 8 — Repair sets, surfaces, sections, loads, and BCs

Deleting the original elements and adding a new region can break references:

1. **Sections:** ensure the new region has a section assignment (same material/section as the original region it replaces) — `Section > Assignment Manager`.
2. **Element/node sets** that included the deleted region: re-create or edit them so any load/BC/output request that referenced them still resolves. Check the Set and Surface managers for empty or partial sets.
3. **Loads** / **BCs** / **interactions** / **output requests** that acted on the deleted region: reassign them to the appropriate faces/sets on the

new region (or on the updated original surface). This is the most common source of a changed-but-wrong result — verify each one against the original intent.
4. **Contact** / **other interactions** touching the modified region: confirm surfaces still contain the intended faces.

Step 9 — Write the input and sanity-check it

1. `Job > Create`, then `Job > Write Input` (don't submit yet).
2. Open the generated `.inp` and confirm:
    - The new region's `*Node` / `*Element` labels are all ≥ 20001 and there are **no duplicate labels** (Abaqus errors on duplicates at datacheck — a good backstop).
    - A single `*Tie` constraint with the correct main/secondary surfaces.
    - Loads, BCs, step, and output requests are intact.
3. Run a **datacheck** first: submit with `Job > ...` datacheck only (or add `*PREPRINT` and run datacheck). Resolve any warnings about tie surfaces, unused nodes, or missing sections.
4. If datacheck is clean, submit the full analysis.

Step 10 — Post-run verification

- Check the `.dat` / `.msg` for tie-related warnings (e.g., many secondary nodes adjusted, or nodes not tied).
- Visualize the interface: contour stress/displacement across the tie. A well-formed tie shows continuous displacement and reasonable (not wildly discontinuous) stress across the interface. Some stress jump at a tie between dissimilar densities is normal; a large unphysical jump suggests a tolerance or surface-selection problem.
- Compare a global quantity (reaction, total mass, a far-field stress) against the original model to confirm you only changed what you intended.

Step 11 — Tracking deleted & added elements and nodes (thermal / convection runs)

When this matters: for a **radiation-only** thermal run you can skip label tracking — surface radiation to a far-field ambient (`*SRADIATE` / cavity radiation) is a property of a *named surface* (emissivity + ambient sink temperature) and never consults individual element or node labels, so redefining the surface on the new faces is enough.

For a run that includes **convection through a user subroutine** (the FILM-type routine driven by your `user_mat` file), it is essential. That subroutine is called at each surface facet and is handed identifiers — element number (`NOEL`), node number (`NODE`) for node-based film, load type (`JLTYP`), and surface name (`SNAME`) — and must return the film coefficient `h` and sink temperature for *that specific facet*. Custom decks branch on those identifiers (or read a table keyed by them). When you delete the old region and rebuild it at 20001+:

- Deleted labels still sitting in the subroutine's convection table now point at facets that no longer exist — ignored at best, mis-applied at worst.
- The new facets (≥ 20001) aren't in the table, so the modified region silently receives **no convection** — a wrong thermal answer with no error raised.

So you must record exactly which labels you removed and which you added, and update the subroutine's convection table accordingly (drop the deleted rows, add the new-label rows with the same physical `h` / `T_sink`).

> **First, confirm what your subroutine keys on.** Open the FORTRAN source behind `user_mat` and search for `NOEL`, `NODE`, `SNAME`, `JLTYP`. Logic like `IF (NOEL .EQ. ...)` / `IF (NODE ...)` or a `READ` of an ID list means it keys on **labels** (you need the full label bookkeeping below). Branching on `SNAME` means it keys on **surface names** (keeping the re-created surfaces' names identical matters more than the underlying labels). Also check the `.inp`: an element-based

`*Surface` under `*SFILM`/`*FILM` ties to element labels; a node-based `*CFILM` ties to node labels.

Dumping a set's labels to a text file

Do this for both the deleted region and the new region. This model is an **assembly of several instances**, so use **assembly-level** sets and record the **instance name alongside each label** — with multiple instances, a bare label is ambiguous (the same part-local label can exist in more than one instance), and the value the user subroutine ultimately sees is the internally-assigned label for that instance's mesh. The 20,000 offset on the new region keeps its labels globally unique, which is exactly what lets the subroutine table address them unambiguously.

Run at the CAE kernel command line (`>>>`). Replace `'Model-1'` and the set names with yours.

Elements:

```
a = mdb.models['Model-1'].rootAssembly
with open('deleted_elems.txt', 'w') as f:
    f.write('instance, label\n')
    for e in a.sets['DEL_REGION_ELEMS'].elements:
        f.write('%s, %d\n' % (e.instanceName, e.label))
```

Nodes:

```
a = mdb.models['Model-1'].rootAssembly
with open('deleted_nodes.txt', 'w') as f:
    f.write('instance, label\n')
    for n in a.sets['DEL_REGION_NODES'].nodes:
        f.write('%s, %d\n' % (n.instanceName, n.label))
```

Repeat both for the new region (`NEW_REGION_ELEMS` / `NEW_REGION_NODES` → `new_elems.txt` / `new_nodes.txt`). Every new label should come back ≥ 20001 — a quick check that the offset took.

> `MeshElement.instanceName` / `MeshNode.instanceName` are populated when the set is read from the **rootAssembly**. If you read from a part-level set instead, `instanceName` is empty — so always pull these from `rootAssembly.sets[...]` in an assembled model.

Keep a mapping table

Maintain one small table alongside the dumps — for each affected facet: instance, old label, surface/face it belonged to, `h` reference, sink-temperature reference, and the new label that replaced it. That table *is* the update your convection subroutine needs: remove the deleted rows, add the new-label rows carrying the same physical `h` / `T_sink`. Cross-check the labels against the `.dat` (from a datacheck run) so you're working with the numbering Abaqus actually assigned after assembly.

Step 12 — Change control across sets, surfaces, cavities, loads, and sections

When the deck carries hundreds of sets and surfaces (e.g. **206 sets and 99 surfaces**) wired into cavity-radiation enclosures and prescribed-temperature boundaries, deleting a region breaks references you cannot reliably find by eye. The failure mode is the same silent one as convection: the job still runs and produces a plausible field. Three things break quietly:

- A **node/element set** that fed a load, BC, or output loses members. If that set is `OUT` or `SINK_I` driving a prescribed temperature, the modified region silently goes **unconstrained** — wrong temperature field, no error.
- A **surface** built on the deleted elements loses its faces. If that surface participates in a **`*CAVITY DEFINITION`**, the enclosure loses radiating area, and the **view factors are now stale** — they must be recomputed, and `VFTOT` re-checked (should be ~1.0 for a closed cavity).
- The new region's elset has **no section assigned**, or (with `TYPE = ORTHO` conductivity) inherits the wrong `*ORIENTATION` and

applies conductivity on global Cartesian axes instead of the cylindrical frame.

So treat this as formal change control: enumerate every impacted entity, record the required action, do it, and verify — before submitting.

Automate the "what's impacted" step

Do not hand-scan 206 sets. Run **`inp_impact_report.m`** on the pre-swap `.inp` with your deleted-label lists from Step 11:

```
inp_impact_report('stress_relief_cooldown.inp', ...
                  'deleted_elems.txt', 'deleted_nodes.txt
```

It parses `*NODE` / `*ELEMENT` / `*NSET` / `*ELSET` (incl. `GENERATE`), `*SURFACE`, `*CAVITY DEFINITION`, `*SOLID`/`*SHELL SECTION`, and the reference keywords (`*BOUNDARY`, `*DSLOAD`, `*DFLUX`, `*SFILM`, `*SRADIATE`, `*CFILM`, …), then reports **exactly which** sets, surfaces, cavities, loads/BCs, and sections reference the deleted labels — each with a status (`PARTIAL` / `FULLY_DELETED` / `REFERENCES`) and a recommended action. That CSV is your worklist.

> The parser assumes a **top-level orphan mesh with global numbering** (as in §10.1 of the deck). It warns if it sees `*INSTANCE`, because per-instance repeated labels would need instance-aware keys. Cross-check its labels against the `.dat` from a datacheck run.

Log it in the change-control ledger

Transfer the parser's findings into **`Region_Swap_Change_Control_Ledger.xlsx`** — a tab per entity type (Mesh, Sets, Surfaces, Cavity_Radiation, Loads_BCs, Sections) plus a Summary tab. For every impacted entity record the action, whether the new-region entities were added, and a Verified flag. The Summary tab totals outstanding items; **it must read 0 before you submit.**

Worked examples of the entries (these are the rows to fill):

Entity	Impacted because	Action	Verify
`OUT` (NSET, temp BC)	lost outer-surface nodes in the region	add new-region outer nodes to `OUT`; keep `OP=NEW` once, `OP=MOD` after	new region shows prescribed T in the `.odb`
`SS` (NSET → surface `HEAD`)	region nodes deleted	rebuild set on new faces → rebuild surface → recompute view factors	`VFTOT` ~1.0
`HEAD` (SURFACE in cavity `C15`)	built on deleted element faces	redefine on new tied-region faces; confirm face id (`S#` / `SPOS`/`SNEG`)	radiating area correct
`C15` (`*CAVITY DEFINITION`)	contains impacted surface	**recompute `*RADIATION VIEWFACTORS` for C15**	`VFTOT`, enclosure still opaque-bounded
`*SOLID SECTION` on `FORGING`	region elements deleted	assign same section to new-region elset; `ORIENTATION` = cylindrical	section assigned, orientation cylindrical

Updating the new tied region's sets, surfaces, and section

After the tie is built, positively re-establish the modified region in the model:

1. **Sets:** add the new region's nodes/elements to every impacted set the parser flagged (especially any set feeding a BC, load, cavity surface, or output request). For a prescribed-temperature boundary, the new outer-surface nodes must join `OUT` (or its equivalent) or that face gets no BC.
2. **Surfaces:** redefine each impacted surface on the new region's

element faces with the correct face identifier — `S1`–`S6` for solids, **`SPOS`/`SNEG` for shells** (shell sidedness decides which face radiates; wrong side silently removes radiating area).
3. **Cavity radiation:** for any cavity whose surface you rebuilt, **recompute `*RADIATION VIEWFACTORS`** and re-check `VFTOT`. New/denser faces change the view-factor solve; stale view factors are not carried correctly across the swap.
4. **Section assignment:** assign the section to the new-region elset with the same material, and — because the deck uses `TYPE = ORTHO` conductivity — attach the **cylindrical `*ORIENTATION`**, not the global axes.
5. **Re-run datacheck** and confirm no unassigned-section, empty-set, or unused-node warnings remain, then submit.

Quick reference — tie roles

Side	Surface	Role	Reason
Original orphan mesh (coarser)	exposed free faces after deletion	**Main (master)**	Coarser side; secondary nodes are constrained to it
New modified region (denser)	mating faces of new instance	**Secondary (slave)**	Finer side as secondary → smoothest, most accurate transfer

Common pitfalls

- **Duplicate labels:** if the original mesh has any label ≥ 20000, bump the offset higher (e.g. 100000). Confirm the original max label first.
- **Running the MATLAB script on the wrong file:** only run it on the standalone new-part `.inp`, never the assembled deck.
- **Large initial gap at the tie:** keep the new instance faces close to

the exposed faces; rely on *Adjust secondary initial position* only for small gaps.

- **Forgotten section on the new region:** unassigned section → datacheck error.
- **Broken loads/BCs:** anything that referenced deleted elements/nodes must be reassigned before submitting.
- **Silent loss of convection:** if a FILM-type user subroutine keys on element/node labels and you don't add the new region's labels (≥ 20001) to its table, the modified region gets no convection and the run still completes — a wrong thermal result with no error. Always capture `DEL_REGION_*` / `NEW_REGION_*` sets (Step 11) for convection runs.